

Software Engineering Principles For ML



Suman

29 Jan, 2026 At 8:00 PM

Notes

Trimester-I

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Session 2.3: Software Engineering Principles for ML

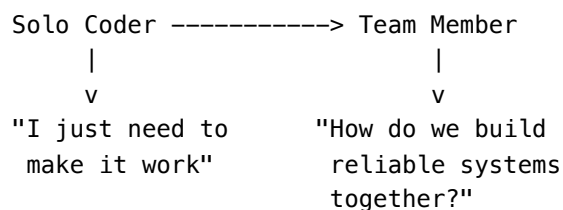
In Class Resource

Program: AIM101 - Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School

Mental Map

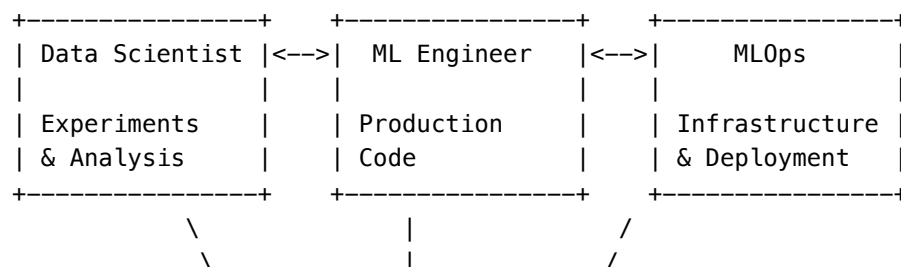
SESSION 2.3: SOFTWARE ENGINEERING PRINCIPLES FOR ML

WHY SE MATTERS



Version Control + Collaboration + Standards

COLLABORATION CULTURE



V
 Communication + Trust + Shared Standards

CODE REVIEW

WHAT WE REVIEW	WHAT WE DON'T
- The CODE	- The PERSON
- The LOGIC	- Their EXPERIENCE
- The APPROACH	- Their VALUE

Learning <-----NOT-----> Gatekeeping

DOCUMENTATION

README	Docstrings	Architecture	Runbooks
"What is this?"	"What does this do?"	"How does it all connect?"	"How do I fix it?"

TECHNICAL DEBT

TODAY	TOMORROW
"Quick fix" (30 minutes)	4 hours to understand + 2 hours to fix
Interest Compounds	

Borrow Time Now -----> Pay Interest Later
(Credit Card Analogy)

AGILE FOR ML

SPRINT 1	SPRINT 2	SPRINT 3
Plan	Plan	Plan
v	v	v
Build	Build	Build
v	v	v
Review	Review	Review
Iterate	Improve	

INDUSTRY MINDSET

ACADEMIA	INDUSTRY
----------	----------

"My code works"	---->	"Our code works
		AND is maintainable
		AND is tested
+-----+		AND is documented"
		+-----+

The Big Picture

You have spent the last few sessions learning Python - variables, data types, control flow, data structures, and functions. You can now write code that works. But here is the uncomfortable truth: **writing code that works is only half the job.**

In the real world, your code will be:

- Read by teammates who did not write it
- Modified six months from now (possibly by you, who will have forgotten everything)
- Deployed to systems where bugs cost real money
- Part of a larger project with dozens of moving pieces

Software Engineering (SE) principles are the practices that transform "code that works on my laptop" into "systems that work reliably in production." These principles become even more critical in Machine Learning, where projects fail not because of bad algorithms, but because of poor engineering practices.

This session is about the **mindset shift** from solo coder to professional team member. The technical skills you have learned are necessary but not sufficient. How you collaborate, communicate, document, and maintain code determines whether your ML projects succeed or fail in the real world.

Section 1: Why Software Engineering Matters for ML

The Solo Coder Illusion

When you learn to code, you typically work alone. You write something, it runs, you move on. This creates a dangerous illusion: the belief that making code work is the goal.

THE SOLO CODER EXPERIENCE

=====

1. Write code
2. Run it
3. Does it work?
 - YES --> Done!
 - NO --> Debug, repeat

Simple, right?

But this model breaks down immediately when you:

- Work with other people
- Need to maintain code over months or years
- Deploy code that affects real users or business decisions
- Scale from a script to a system

What Changes in a Team Environment

SOLO CODING	TEAM CODING
"I understand it"	"Everyone needs to understand it"
"It works"	"It works, AND we can prove it works"
"I remember what I did"	"The code and docs explain what we did"
"I can change it anytime"	"Anyone can change it safely"
"My laptop, my rules"	"Production servers, team standards"

The Unique Challenges of ML Projects

Machine Learning adds layers of complexity that make SE principles even more important:

TRADITIONAL SOFTWARE	ML SOFTWARE
=====	=====
Code + Config	Code + Config
	+
v	Data (huge, evolving)
Output	+
	Models (binary blobs)
	+
	Experiments (dozens)
	+
	Hyperparameters
	v
	Output
More artifacts = More things that can go wrong	
= More things to version	
= More things to document	

Why ML Projects Fail

A landmark 2015 paper from Google titled "Hidden Technical Debt in Machine Learning Systems" revealed that in a typical ML system, the actual ML code is a small fraction of the total system:

THE ML CODE REALITY

=====

```

+-----+
|               |
|  +-----+    |
|  | ML   | <-- This is what courses teach
|  | Code |      (maybe 5% of the system)
|  +-----+    |
|               |
| Configuration | Data Collection | Data Verification | |
| Feature Extraction | Analysis Tools | Process Mgmt   |
| Machine Resource Mgmt | Serving Infrastructure |
| Monitoring | Debugging | Testing | Documentation   |
|               |
| ^-- This is what actually determines success
|      (95% of the system)
|               |
+-----+

```

Most ML courses: "Here's how to build a model"

Reality: "Here's how to build a SYSTEM with a model"

The takeaway is clear: **great ML products fail due to poor SE practices, not bad algorithms.**

Section 2: ML Project Failure Case Studies

Learning from failures is one of the most effective ways to understand why SE principles matter. These are not hypothetical scenarios - they are real incidents that cost real companies real money.

Case Study 1: Google Gorilla (2015)

What Happened:

Google Photos launched an image classification feature that automatically labeled photos. The system classified images of two Black users as "gorillas."

The Root Cause:

This was not a single bug - it was a systematic failure across multiple SE practices:

FAILURES IN THE GOOGLE GORILLA INCIDENT

=====

1. TESTING FAILURE

- Insufficient testing across diverse demographics
- No systematic evaluation of model fairness

2. DATA FAILURE

- Training data was not representative
- No data documentation explaining dataset composition

3. REVIEW FAILURE

- No human review process for sensitive categories
- Automated systems deployed without guardrails

4. MONITORING FAILURE

- No automated detection of problematic classifications
- Users had to report the issue externally

SE Lessons:

Principle Violated	What Should Have Been Done
Comprehensive Testing	Test models on diverse, representative data
Data Documentation	Maintain "data cards" describing dataset demographics
Human-in-the-Loop	Add human review for high-risk classification categories
Monitoring	Build automated alerts for anomalous or sensitive outputs

Google's "fix" was to simply remove "gorilla" as a possible label - not a real solution, but a testament to how hard the underlying problem was to solve properly.

Case Study 2: Knight Capital (2012)

What Happened:

Knight Capital, one of the largest trading firms in the US, deployed new trading software. Within 45 minutes, they lost \$440 million - more than the company was worth. The company was bankrupt by the end of the week.

The Root Cause:

THE KNIGHT CAPITAL DISASTER – TIMELINE

=====

1. OLD CODE PROBLEM

- Eight servers needed the new code
- A technician forgot to deploy to ONE server
- That server still had old, obsolete code

2. CODE CONFLICT

- New code: Execute trades normally
- Old code: Execute trades in a deprecated "test mode"
- Test mode: Buy high, sell low (opposite of intended)

3. NO SAFEGUARDS

- No version verification before launch
- No automated deployment process
- No rollback procedure ready
- No circuit breakers to stop runaway trading

4. RESULT

- \$440 million lost in 45 minutes
- Company bankrupt
- 440 jobs lost

SE Lessons:

Principle Violated	What Should Have Been Done
Version Control	Track and verify what code is running on each server
Deployment Automation	Use automated, repeatable deployment processes
Testing	Verify deployments in staging before production
Circuit Breakers	Build automatic safeguards that halt when anomalies detected

Case Study 3: Zillow iBuying (2021)**What Happened:**

Zillow used ML models to predict home prices and automatically purchase homes for resale. In November 2021, Zillow announced it was shutting down the program after losing \$881 million and laying off 2,000 employees.

The Root Cause:

THE ZILLOW iBUYING FAILURE

=====

1. MODEL OVERCONFIDENCE

- Models predicted prices with false precision
- No uncertainty quantification
- "The model says \$500,000" vs "The model says \$480,000-\$520,000"

2. FEEDBACK LOOP BLINDNESS

- Zillow's own purchases affected the market
- Model was trained on historical data
- Did not account for their own market impact

3. REGIME CHANGE

- COVID-19 fundamentally changed housing market
- Models trained on pre-COVID data
- No monitoring for distribution shift

4. HUMAN OVERRIDE SUPPRESSED

- Acquisition teams pressured to follow model
- Local knowledge ignored
- "Trust the algorithm" culture

5. NO CIRCUIT BREAKERS

- No automatic pause when losses exceeded threshold
- Kept buying as losses mounted

SE Lessons:

Principle Violated	What Should Have Been Done
Uncertainty Quantification	Models should output confidence intervals, not point estimates
Monitoring for Drift	Detect when model inputs no longer match training distribution
Human-in-the-Loop	Enable and value human override of model decisions
Feedback Loop Analysis	Understand how model outputs affect future inputs
Circuit Breakers	Automatic halt when KPIs exceed safety thresholds

The Common Thread

All three case studies share a common pattern:

THE FAILURE PATTERN

=====

1. Technical capability existed
(The code/model technically "worked")
2. Software engineering practices were weak
(Testing, monitoring, documentation, review)
3. Failure modes were predictable
(Experts could have identified risks)
4. Costs were enormous
(Money, jobs, reputation, harm to users)

The lesson: SE practices are not bureaucratic overhead. They are the difference between success and catastrophic failure.

Section 3: Collaboration Culture in Tech Teams

Roles in an ML Team

Modern ML projects require diverse skills that no single person possesses. Understanding team roles helps you collaborate effectively.

ML TEAM ROLES

=====

ROLE	RESPONSIBILITIES
Data Scientist	- Explore data, find patterns - Build and evaluate models - Communicate insights to stakeholders - Focus: "Does this model solve the business problem?"
ML Engineer	- Turn prototype models into production - Optimize for speed and efficiency - Build training pipelines - Focus: "Can this model run reliably at scale?"
Data Engineer	- Build data pipelines - Ensure data quality and availability - Manage data infrastructure - Focus: "Is the right data in the right place at the right time?"
MLOps/Platform	- Deploy and monitor models - Manage infrastructure - Build automation and tooling - Focus: "Is this model running correctly in production?"
Backend Engineer	- Build APIs and services - Integrate ML into products - Handle non-ML system components - Focus: "How does the model fit into the larger system?"
Product Manager	- Define requirements and priorities - Translate business needs to technical - Manage timelines and stakeholders - Focus: "What should we build and why?"

Research vs Production Tensions

One of the most common sources of friction in ML teams is the tension between research mindset and production mindset:

RESEARCH MINDSET

=====

"Let me try this"

"Accuracy is everything"

"I got 0.5% improvement!"

"The notebook works"

PRODUCTION MINDSET

=====

vs. "Will this work at scale?"

vs. "Accuracy, latency, cost, reliability all matter"

vs. "Is 0.5% worth the complexity?"

vs. "Is it tested, documented, monitored?"

"New state-of-the-art!" vs. "Can we actually deploy this?"

"Let me explore freely" vs. "We have a deadline"

Neither mindset is wrong - they are both necessary. The key is understanding when each applies:

WHEN EACH MINDSET APPLIES

=====

RESEARCH MINDSET APPROPRIATE:

- Early exploration phase
- Proof of concept
- Academic publications
- Understanding what's possible

PRODUCTION MINDSET APPROPRIATE:

- Deploying to real users
- Systems with business impact
- Long-term maintenance
- Scaling beyond prototype

Building Trust in Teams

High-performing teams are built on psychological safety - the belief that you will not be punished for making mistakes or asking questions.

SIGNS OF PSYCHOLOGICAL SAFETY

=====

- + Team members admit mistakes openly
- + Questions are welcomed, not dismissed
- + Diverse opinions are sought out
- + Failures are analyzed, not blamed
- + Help is requested and offered freely
- + Credit is shared generously

SIGNS OF PSYCHOLOGICAL DANGER

=====

- Mistakes are hidden or blamed on others
- Questions are treated as weakness
- Disagreement is seen as conflict
- Failures lead to finger-pointing
- People work in silos to avoid exposure
- Credit is hoarded, blame is deflected

Communication Patterns That Work

EFFECTIVE COMMUNICATION IN TECH TEAMS

=====

1. ASSUME POSITIVE INTENT

"Why did you do it this way?"

Read as: Curious, want to learn
NOT as: Accusation, criticism

2. MAKE THE IMPLICIT EXPLICIT

Bad: "Fix the thing"

Good: "Please fix the timeout bug in the login service tracked in issue #42"

3. DOCUMENT DECISIONS

Bad: "We decided in the meeting"

Good: "Decision documented in Confluence/Notion page X"

4. OVER-COMMUNICATE UNCERTAINTY

Bad: "This might not work" (then ship it)

Good: "I have concerns about X, Y, Z. Can we discuss before deployment?"

5. GIVE CONTEXT, NOT JUST TASKS

Bad: "Build a dashboard"

Good: "We need a dashboard so the ops team can monitor model latency. Here's what they need to see..."

Section 4: Code Review Philosophy

Code review is one of the most important SE practices, yet it is frequently misunderstood and poorly executed.

Why We Review Code

THE FOUR PILLARS OF CODE REVIEW

=====

CATCH BUGS	SHARE KNOWLEDGE
Fresh eyes spot what author missed	Reviewer learns new code areas
IMPROVE QUALITY	MAINTAIN CONSISTENCY
"Good enough" becomes "better"	Team uses same patterns/styles

But there is a deeper reason: **code review is a mentorship mechanism**. Senior developers guide juniors. Juniors ask questions that make seniors reconsider assumptions. Everyone improves.

The Golden Rule

```

+-----+
|               |
| REVIEW THE CODE, NOT THE PERSON |
|               |
+-----+

```

This distinction is CRITICAL:

REVIEWING CODE

=====

"This function could
be clearer"

"What happens if X
is null?"

"Consider approach Y"

"I found a bug here"

REVIEWING PERSON

=====

"You wrote confusing
code"

"You didn't think about
edge cases"

"You should have done
it this way"

"You made a mistake"

The difference is subtle but profound. When you review code, you are helping improve a shared artifact. When you review the person, you are making a judgment about their competence.

Good vs Bad Review Comments

Bad Comment	Why It Is Bad	Good Comment	Why It Is Good
"This is wrong"	No explanation, no path forward	"This might fail when the list is empty. Consider adding a check for that case."	Specific problem, suggests solution
"Why did you do it this way?"	Sounds accusatory	"I'm curious about this approach. Could you help me understand the reasoning?"	Shows curiosity, invites dialogue
"This is ugly"	Subjective, vague	"Consider renaming <code>x</code> to <code>user_count</code> - it would make line 45 clearer."	Specific, actionable
"nope"	Dismissive, unhelpful	"This approach has a performance issue with large datasets. Here's an alternative that scales better."	Explains problem, offers alternative
"..."	Passive-aggressive	"This works, but have you considered X for maintainability?"	Acknowledges good, suggests improvement

The Two-Way Street

Code review is a relationship, not a transaction:

TODAY
=====

TOMORROW
=====

You are the author <---> You are the reviewer
You receive feedback You give feedback

The culture you create is the culture you experience.

If you write harsh reviews --> Expect harsh reviews
If you are constructive --> Others will be constructive
If you explain reasoning --> Others will explain
If you express gratitude --> Others will appreciate you

Receiving Review Feedback

Being reviewed can feel vulnerable. Here is how to handle it professionally:

RECEIVING FEEDBACK WELL
=====

1. ASSUME POSITIVE INTENT
The reviewer is trying to help, not attack
2. SEPARATE EGO FROM CODE
The code is not you. Criticism of code is not criticism of you.
3. RESPOND TO ALL COMMENTS
Even if just "Good point, fixed!" or "I disagree because X"
4. DISAGREE RESPECTFULLY
"I considered that approach. Here's why I went another way..."
5. EXPRESS GRATITUDE
"Thanks for catching that!" costs nothing and builds relationships
6. LEARN FROM PATTERNS
If multiple reviewers flag the same issue, pay attention

Section 5: Documentation as Leadership

Documentation is often seen as a chore - something you do after the "real work" is done. This is backwards. **Documentation is leadership**: it shows you care about the future, about your teammates, about the project's success beyond your own involvement.

Types of Documentation

DOCUMENTATION HIERARCHY
=====

+-----+

	README		<-- "What is this project?"
+-----+		+-----+	First thing people see
+-----+		+-----+	
	DOCSTRINGS		<-- "What does this function do?"
+-----+		+-----+	In the code itself
+-----+		+-----+	
	ARCHITECTURE		<-- "How does it all fit together?"
	DOCS		System design decisions
+-----+		+-----+	
+-----+		+-----+	
	RUNBOOKS		<-- "How do I operate this?"
+-----+		+-----+	Deployment, debugging, incidents

README: Your Project's Front Door

The README is the first thing anyone sees. It should answer:

README ESSENTIALS

=====

1. WHAT IS THIS?
One-paragraph description of what the project does
2. HOW DO I GET STARTED?
Installation/setup steps (ideally copy-paste-able)
3. HOW DO I USE IT?
Basic usage examples
4. HOW DO I CONTRIBUTE?
For projects accepting contributions
5. WHO MAINTAINS THIS?
Contact info or team ownership

Docstrings: Self-Documenting Code

Docstrings explain what functions/classes do. They live in the code itself:

```
def calculate_customer_lifetime_value(
    customer_transactions: list,
    discount_rate: float = 0.1
) -> float:
    """
    Calculate the customer lifetime value (CLV) from transaction history.

    CLV represents the total revenue expected from a customer over their
    entire relationship with the business.

    Parameters
    -----
    customer_transactions : list
```

```
List of transaction amounts (positive floats)
discount_rate : float, optional
    Annual discount rate for future value (default: 0.1)
```

Returns

```
-----
```

```
float
```

```
    Estimated lifetime value in dollars
```

Examples

```
-----
```

```
>>> calculate_customer_lifetime_value([100, 150, 200])
450.0
>>> calculate_customer_lifetime_value([100, 150], discount_rate=0.05)
238.10
.....
```

ML-Specific Documentation

ML projects need documentation that traditional software does not:

ML DOCUMENTATION REQUIREMENTS

```
=====
```

1. EXPERIMENT LOGS

- What did you try?
- What parameters did you use?
- What were the results?
- Why did you make the choices you made?

2. MODEL CARDS

- What does this model do?
- What data was it trained on?
- What are its known limitations?
- When should it NOT be used?
- What fairness/bias considerations apply?

3. DATA DOCUMENTATION

- Where does this data come from?
- How was it collected?
- What preprocessing was applied?
- What is NOT in the data?
- When was it last updated?

4. PIPELINE DOCUMENTATION

- What are the inputs and outputs?
- What are the dependencies?
- How long does it take to run?
- What can fail and how to fix it?

The "Hit by a Bus" Test

Ask yourself: **If I got hit by a bus tomorrow, could my team continue this project?**

THE "HIT BY A BUS" CHECKLIST =====

Could someone else:

- [] Find the code repository?
- [] Set up the development environment?
- [] Understand what the project does?
- [] Run the tests?
- [] Deploy a new version?
- [] Debug common issues?
- [] Contact someone for help?

If any answer is "no," you have a documentation problem.

Section 6: Technical Debt

The Credit Card Analogy

Technical debt is the most intuitive concept in software engineering once you understand the analogy:

THE CREDIT CARD ANALOGY =====

FINANCIAL DEBT =====

You want to buy something
you cannot afford right now

|
v

Use a credit card
(borrow money)

|
v

You get what you want NOW

|
v

You owe money LATER

|
v

Interest accumulates
(cost grows over time)

TECHNICAL DEBT =====

You need to ship a feature
but don't have time to do
it "right"

|
v

Take a shortcut
(skip tests, documentation,
proper design)

|
v

Feature ships NOW

|
v

Code is harder to maintain
LATER

|
v

Problems compound
(more shortcuts = more mess)

Just like financial debt, technical debt is not always bad. Sometimes borrowing is the right choice:

- Emergency fixes
- Time-sensitive opportunities

- Proof of concept before investing fully

But also like financial debt, **interest compounds**. A little debt is manageable. A lot of debt becomes overwhelming.

How Technical Debt Accumulates in ML

ML projects are especially prone to technical debt:

ML TECHNICAL DEBT PATTERNS

=====

1. EXPERIMENT CODE --> PRODUCTION
"I'll clean this up later"
(Narrator: They did not clean it up later)
2. UNDOCUMENTED ASSUMPTIONS
"The model assumes X is always positive"
(Six months later: "Why is the model failing?")
3. COPY-PASTE REUSE
Same preprocessing code in 5 notebooks
Bug in one means bugs in all
4. DEAD CODE ACCUMULATION
"I might need this later"
(Narrator: They never needed it, and now no one knows if it's safe to delete)
5. CONFIGURATION SPRAWL
Parameters hardcoded everywhere
No single source of truth
6. TEST AVOIDANCE
"ML is inherently stochastic"
(This is not a valid excuse)
7. DATA PIPELINE FRAGILITY
"Works on my machine with this data"
Breaks when data format changes slightly

Signs of Too Much Debt

WARNING SIGNS

=====

1. FEAR OF CHANGE
"Don't touch that code, it might break everything"
When no one dares to modify code, debt has accumulated
2. TRIBAL KNOWLEDGE
"Only Alice understands the training pipeline"
Critical knowledge in one person's head = massive risk
3. RECURRING BUGS

Same issues keep coming back
Root causes never addressed

4. ONBOARDING PAIN

New team members take months to be productive
Code is impenetrable without oral history

5. DEPLOYMENT DREAD

Every release feels risky
No confidence that changes won't break production

6. VELOCITY DECLINE

"Why does everything take so long now?"
Debt makes every change harder

Managing Technical Debt

DEBT MANAGEMENT STRATEGIES

=====

1. MAKE IT VISIBLE

Track debt as actual work items
"Refactor payment module" is a real task

2. BUDGET DEBT PAYMENT

Allocate time each sprint to pay down debt
20% maintenance is common rule of thumb

3. BOY SCOUT RULE

"Leave the code better than you found it"
Small improvements compound over time

4. PREVENT NEW DEBT

Code review catches shortcuts
Standards prevent common issues

5. DOCUMENT KNOWN DEBT

At minimum, acknowledge it exists
"TODO: This is a hack because X. Should fix when Y."

Section 7: Agile Basics for ML Teams

What is Agile?

Agile is a set of principles for managing work, originally from software development but now applied broadly. The core idea: **embrace change, deliver incrementally, learn continuously**.

AGILE VS WATERFALL

=====

WATERFALL (Old Way)

=====

AGILE (Modern Way)

=====

Plan everything upfront

|
v

Build everything

|
v

Test at the end

|
v

Deliver once

|
v

"Surprise! Here's what
you asked for 18 months
ago!"

Plan incrementally

|
v

Build a small piece

|
v

Test continuously

|
v

Deliver frequently

|
v

"Here's what we have so far.
What do you think?
Let's adjust."

Core Agile Practices

SPRINT

=====

A fixed time period (usually 2 weeks) for completing work.

Week 1

Week 2

Work on planned tasks	Finish work Review & demo
-----------------------	------------------------------

|
v

Start next sprint

STANDUP

=====

Daily check-in (15 minutes max). Three questions:

1. What did I do yesterday?
2. What will I do today?
3. Any blockers?

RETROSPECTIVE

=====

End-of-sprint reflection. Three questions:

1. What went well?
2. What could be improved?
3. What will we try differently?

BACKLOG

=====

Prioritized list of all work to be done.

#	TASK	PRIORITY
---	------	----------

- Document findings
- Update metrics dashboards
- Code review
- Prepare demo

DAY 10: SPRINT REVIEW

- Demo progress to stakeholders
- Discuss results

DAY 10: RETROSPECTIVE

- What worked?
- What to improve?
- Action items

Section 8: Industry Best Practices

Big Tech vs Startups

Different company types have different approaches to SE practices:

BIG TECH

=====

Extensive code review
(hours of review per change)

Comprehensive testing
(90%+ coverage required)

Detailed documentation
(formal design docs)

Slow, careful releases
(staged rollouts)

Many specialists
(clear role boundaries)

STARTUP

=====

"Ship it, fix later"
(speed over perfection)

"Manual testing is fine"
(limited testing resources)

"The code IS the documentation"
(tribal knowledge)

"Deploy 10x per day"
(fix in production)

Everyone does everything
(generalists required)

Neither approach is universally "right." But as a new practitioner, you should:

- **At startups:** Push for more process than seems necessary (you will thank yourself later)
- **At big tech:** Understand WHY processes exist (they exist for good reasons)

Academia vs Industry

If you are coming from an academic background (or building skills for industry), the mindset shift is significant:

ACADEMIA

=====

INDUSTRY

=====

GOAL: Advance knowledge
(Novelty matters most)

GOAL: Deliver value
(Reliability matters most)

TIMELINE: Paper deadline
(Sprint to submit)

TIMELINE: Business needs
(Sustainable pace)

SUCCESS: Publication accepted
(Peer review)

SUCCESS: Users happy, system stable
(Business metrics)

CODE: Proof of concept
(Works once = done)

CODE: Production-ready
(Works always = done)

COLLABORATION: Mostly solo
(First author = main contributor)

COLLABORATION: Team sport
(Shared ownership)

FAILURE: Rejected paper
(Academic setback)

FAILURE: Outage, lost revenue
(Real consequences)

What Hiring Managers Look For

When you interview for ML/software roles, technical skills are necessary but not sufficient. Here is what sets candidates apart:

WHAT HIRING MANAGERS EVALUATE

=====

TECHNICAL SKILLS (Table Stakes)

- Can you code?
- Do you understand ML concepts?
- Can you solve problems?

PROFESSIONAL SKILLS (Differentiators)

- Can you write clean, readable code?
- Do you write tests?
- Can you document your work?
- Do you handle feedback well?
- Can you explain complex ideas simply?
- Do you think about edge cases?
- Do you consider maintainability?

COLLABORATION SIGNALS

- How do you talk about past teammates?
- How do you handle disagreement?
- Do you give credit to others?
- Can you admit what you don't know?

The Portfolio Signal

Your GitHub/portfolio tells a story:

STRONG PORTFOLIO SIGNALS

=====

Clean, documented README
Tests included

WEAK PORTFOLIO SIGNALS

=====

No README
No tests

Code is organized
Commits have clear messages
Project has clear purpose
Dependencies documented

Single giant file
"asdf", "fix", "stuff"
Unclear what it does
"Works on my machine"

Key Takeaways

- +-----+
| KEY TAKEAWAYS |
+-----+
1. GREAT ML PRODUCTS FAIL DUE TO POOR SE PRACTICES, NOT BAD ALGORITHMS
The code is often a small part of the system.
Engineering practices determine success.
 2. YOU ARE JOINING A TEAM, NOT CODING ALONE
Understanding team roles, communication, and collaboration
is as important as technical skills.
 3. CODE REVIEW IS ABOUT LEARNING, NOT JUDGMENT
Review the code, not the person.
Both giving and receiving feedback are skills to develop.
 4. DOCUMENTATION IS LEADERSHIP
It shows you care about the future.
The "hit by a bus" test: could someone else continue?
 5. TECHNICAL DEBT COMPOUNDS LIKE CREDIT CARD INTEREST
Shortcuts today cost more tomorrow.
Make debt visible, budget time to pay it down.
 6. AGILE HELPS MANAGE ML'S INHERENT UNCERTAINTY
Work in sprints. Embrace iteration.
Adapt practices for ML's unique needs.
 7. INDUSTRY VALUES SE SKILLS AS MUCH AS ML KNOWLEDGE
Clean code, tests, documentation, collaboration.
These are what separate juniors from seniors.
- +-----+

Quick Reference: Key Terms

Term	Definition
Technical Debt	Shortcuts taken now that cost more to fix later. Like credit card debt for code.
Sprint	A fixed time period (usually 2 weeks) for completing a set of planned work.

Term	Definition
Standup	Brief daily meeting where team members share progress and blockers.
Backlog	Prioritized list of all work to be done on a project.
Retrospective	End-of-sprint meeting to reflect on what worked and what to improve.
Code Review	Peer examination of code before it is merged. Catches bugs, shares knowledge.
Model Card	Documentation of what an ML model does, its training data, and its limitations.
Psychological Safety	Team environment where it is safe to take risks and admit mistakes.
Agile	Methodology emphasizing iterative development, collaboration, and adaptability.
CI/CD	Continuous Integration/Continuous Deployment - automated testing and deployment.

Common Mistakes to Avoid

COMMON MISTAKES AND HOW TO AVOID THEM

=====

MISTAKE: "I'll document it later"

FIX: Document as you go. Later never comes.

MISTAKE: "Tests slow me down"

FIX: Tests save time by catching bugs early. Investment, not cost.

MISTAKE: "My code is self-explanatory"

FIX: It's not. Write docstrings and comments for your future self.

MISTAKE: Taking code review personally

FIX: Separate your ego from your code. Feedback improves the work.

MISTAKE: "Only I need to understand this"

FIX: Code is read more than written. Others will need to understand.

MISTAKE: Ignoring technical debt

FIX: Track it explicitly. Budget time to address it.

MISTAKE: Skipping retrospectives

FIX: Continuous improvement requires reflection. Make time for it.

Preview: Week 3 Hands-On Git

In Week 3, you will put these principles into practice with hands-on Git training:

WEEK 3 PREVIEW

=====

- Installing and configuring Git
- Creating your first repository
- Making commits with good messages
- Creating and merging branches
- Setting up GitHub
- Making your first pull request
- Experiencing code review firsthand

Today was the WHY.

Week 3 is the HOW.

The concepts you learned today - collaboration, code review, documentation - will all come alive when you start working with version control tools.

Self-Assessment Checklist

Before moving on, verify you understand:

- [] Why SE practices matter more for ML than traditional software
- [] The roles in an ML team and how they collaborate
- [] The golden rule of code review
- [] How to give and receive constructive feedback
- [] The different types of documentation and their purposes
- [] What technical debt is and why it matters
- [] Basic Agile concepts: sprints, standups, retrospectives
- [] How industry expectations differ from academic expectations

If any of these are unclear, review the relevant section before Week 3.

Post Class Content: [Session_2_3_Post_read_and_Resources](#)

Vishlesan i-Hub IIT Patna x Masai School